

# Simple Commander API

## Overview

The goal of the Nav2 Simple (Python3) Commander is to provide a “navigation as a library” capability to Python3 users. We provide an API that handles all the ROS 2 and Action Server tasks for you such that you can focus on building an application leveraging the capabilities of Nav2 (after you’ve configured it to your liking with your plugins of choice). [We also provide you with demos and examples of API usage](#) to build common basic capabilities in autonomous mobile robotics in the `nav2_simple_commander` package.

A simple demonstration is shown below. Note: `goToPose()`, `goThroughPoses()`, `followWaypoints()` and similar are **non-blocking** such that you can receive and process feedback in a single-threaded application. As such while waiting for a task to be completed, the `while not nav.isTaskComplete()` design is necessary to poll for changes in the navigation completion, and if not complete some tasks of interest to your application (like processing feedback, doing something with the data the robot is collecting, or checking for faults).

You may use this simple commander preempt commands of the same type (e.g. you can preempt a `goToPose()` with another `goToPose()`) but you must explicitly cancel a current command and issue a new one if switching between `goToPose()`, `goThroughPoses()`, or `followWaypoints()`.

```

from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
import rclpy

rclpy.init()
nav = BasicNavigator()

# ...

nav.setInitialPose(init_pose)
nav.waitUntilNav2Active() # if autostarted, else use lifecycleStartup()

# ...

path = nav.getPath(init_pose, goal_pose)
smoothed_path = nav.smoothPath(path)

# ...

nav.goToPose(goal_pose)
while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback.navigation_duration > 600:
        nav.cancelTask()

# ...

result = nav.getResult()
if result == TaskResult.SUCCEEDED:
    print('Goal succeeded!')
elif result == TaskResult.CANCELED:
    print('Goal was canceled!')
elif result == TaskResult.FAILED:
    print('Goal failed!')

```

## Commander API

The methods provided by the basic navigator are shown below, with inputs and expected returns. If a server fails, it may throw an exception or return a *None* object, so please be sure to properly wrap your navigation calls in try/catch and check returns for *None* type.

New as of September 2023: the simple navigator constructor will accept a *namespace* field to support multi-robot applications or namespaced Nav2 launches.

| Robot Navigator Method                  | Description                                                                             |
|-----------------------------------------|-----------------------------------------------------------------------------------------|
| setInitialPose(initial_pose)            | Sets the initial pose ( <code>PoseStamped</code> ) of the robot to localization.        |
| goThroughPoses(poses, behavior_tree="") | Requests the robot to drive through a set of poses (list of <code>PoseStamped</code> ). |
| goToPose(pose, behavior_tree="")        | Requests the robot to drive to a pose ( <code>PoseStamped</code> ).                     |

| Robot Navigator Method                                                                             | Description                                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| followWaypoints(poses)                                                                             | Requests the robot to follow a set of waypoints (list of <code>PoseStamped</code> ). This will execute the chosen <code>TaskExecutor</code> plugin at each pose.                                                                           |
| followPath(path, controller_id="", goal_checker_id="", progress_checker_id="", path_handler_id="") | Requests the robot to follow a path from a starting to a goal <code>PoseStamped</code> , <code>nav_msgs/Path</code> .                                                                                                                      |
| spin(spin_dist=1.57, time_allowance=10, disable_collision_checks=False)                            | Requests the robot to performs an in-place rotation by a given angle.                                                                                                                                                                      |
| driveOnHeading(dist=0.15, speed=0.025, time_allowance=10, disable_collision_checks=False)          | Requests the robot to drive on heading by a given distance.                                                                                                                                                                                |
| backup(backup_dist=0.15, backup_speed=0.025, time_allowance=10, disable_collision_checks=False)    | Requests the robot to back up by a given distance.                                                                                                                                                                                         |
| assistedTeleop(time_allowance=30)                                                                  | Requests the robot to run the assisted teleop action.                                                                                                                                                                                      |
| cancelTask()                                                                                       | Cancel an ongoing task, including route tasks.                                                                                                                                                                                             |
| isTaskComplete(task=RunningTask.NONE)                                                              | Checks if task is complete yet, times out at <code>100ms</code> . Returns <code>True</code> if completed and <code>False</code> if still going. Provide the task ID from the long-running task (follow path, compute and track route, etc) |
| getFeedback(task=RunningTask.NONE)                                                                 | Gets feedback from task, returns action server feedback msg. provide the task ID for the task you are requesting.                                                                                                                          |
| getResult()                                                                                        | Gets final result of task, to be called after <code>isTaskComplete</code> returns <code>True</code> . Returns action server result msg.                                                                                                    |
| getPath(start, goal, planner_id="", use_start=False)                                               | Gets a path from a starting to a goal <code>PoseStamped</code> , <code>nav_msgs/Path</code> .                                                                                                                                              |
| getPathThroughPoses(start, goals, planner_id="", use_start=False)                                  | Gets a path through a starting to a set of goals, a list of <code>PoseStamped</code> , <code>nav_msgs/Path</code> .                                                                                                                        |
| getRoute(start, goal, use_start=False)                                                             | Gets a route from a set start and goal nodeIDs or <code>PoseStamped</code> . Use Start if given, otherwise uses TF to obtain robot pose.                                                                                                   |
| getAndTrackRoute(start, goal, use_start=False)                                                     | Gets a route from a set of start and goal NodeIDs or <code>PoseStamped</code> . Uses start if given, otherwise uses TF to obtain the robot pose.                                                                                           |
| dockRobot(dock_pose, dock_type)                                                                    | Attempts to dock the robot at a given docking pose and type, without using docking database of known docks.                                                                                                                                |
| dockRobot(dock_id)                                                                                 | Attempts to dock the robot at a given dock ID in the database of known docks.                                                                                                                                                              |

| Robot Navigator Method                                                                     | Description                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>undockRobot(dock_type='')</code>                                                     | Undocks robot. If docking server instance was used to dock, type is not required.                                                                                                                                 |
| <code>smoothPath(path, smoother_id='', max_duration=2.0, check_for_collision=False)</code> | Smooths a given path of type <code>nav_msgs/Path</code> .                                                                                                                                                         |
| <code>changeMap(map_filepath)</code>                                                       | Requests a change from the current map to <i>map_filepath</i> 's yaml.                                                                                                                                            |
| <code>clearAllCostmaps()</code>                                                            | Clears both the global and local costmaps.                                                                                                                                                                        |
| <code>clearLocalCostmap()</code>                                                           | Clears the local costmap.                                                                                                                                                                                         |
| <code>clearGlobalCostmap()</code>                                                          | Clears the global costmap.                                                                                                                                                                                        |
| <code>clearLocalCostmapAroundPose(PoseStamped, distance)</code>                            | Clears the local costmap around given pose.                                                                                                                                                                       |
| <code>clearGlobalCostmapAroundPose(PoseStamped, distance)</code>                           | Clears the global costmap around given pose.                                                                                                                                                                      |
| <code>clearCostmapExceptRegion( distance)</code>                                           | Clears the local costmap around current robot pose.                                                                                                                                                               |
| <code>getGlobalCostmap()</code>                                                            | Returns the global costmap, <code>nav2_msgs/Costmap</code> .                                                                                                                                                      |
| <code>getLocalCostmap()</code>                                                             | Returns the local costmap, <code>nav2_msgs/Costmap</code> .                                                                                                                                                       |
| <code>toggleCollisionMonitor(enable)</code>                                                | Toggles the collision monitor on ( <i>True</i> ) or off ( <i>False</i> ).                                                                                                                                         |
| <code>followObjectByTopic(topic, max_duration=0)</code>                                    | Requests the robot to follow an object by subscribing to a given topic.                                                                                                                                           |
| <code>followObjectByFrame(frame, max_duration=0)</code>                                    | Requests the robot to follow an object by a given frame.                                                                                                                                                          |
| <code>waitUntilNav2Active(navigator='bt_navigator', localizer='amcl')</code>               | Blocks until Nav2 is completely online and lifecycle nodes are in the active state. To be used in conjunction with autostart or external lifecycle bringup. Custom navigator and localizer nodes can be specified |
| <code>lifecycleStartup()</code>                                                            | Sends a request to all lifecycle management servers to bring them into the active state, to be used if autostart is <code>False</code> and you want this program to control Nav2's lifecycle.                     |
| <code>lifecycleShutdown()</code>                                                           | Sends a request to all lifecycle management servers to shut them down.                                                                                                                                            |
| <code>destroyNode()</code>                                                                 | Releases the resources used by the object.                                                                                                                                                                        |

## Costmap API

This is a Python3 API for costmap 2d messages from the stack. It provides the basic conversion, get/set, and handling semantics found in the costmap 2d C++ API.

| Costmap Method              | Description                                                                                                                                                                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| getSizelnCellsX()           | Get map width in cells.                                                                                                                                                                                                             |
| getSizelnCellsY()           | Get map height in cells.                                                                                                                                                                                                            |
| getSizelnMetersX()          | Get x axis map size in meters.                                                                                                                                                                                                      |
| getSizelnMetersY()          | Get y axis map size in meters.                                                                                                                                                                                                      |
| getOriginX()                | Get the origin x axis of the map [m].                                                                                                                                                                                               |
| getOriginY()                | Get the origin y axis of the map [m].                                                                                                                                                                                               |
| getResolution()             | Get map resolution [m/cell].                                                                                                                                                                                                        |
| getGlobalFrameID()          | Get global frame_id.                                                                                                                                                                                                                |
| getCostmapTimestamp()       | Get costmap timestamp.                                                                                                                                                                                                              |
| getCostXY(mx, my)           | Get the cost ( <code>np.uint8</code> ) of a cell in the costmap using mx ( <code>int</code> ), my ( <code>int</code> ) of Map Coordinate.                                                                                           |
| getCostIdx(index)           | Get the cost ( <code>np.uint8</code> ) of a cell in the costmap using index ( <code>int</code> )                                                                                                                                    |
| setCost(mx, my, cost)       | Set the cost ( <code>np.uint8</code> ) of a cell in the costmap using mx ( <code>int</code> ), my ( <code>int</code> ) of Map Coordinate.                                                                                           |
| mapToWorld(mx, my)          | Get the wx ( <code>float</code> ) [m], wy ( <code>float</code> ) [m] of world coordinate XY using mx ( <code>int</code> ), my ( <code>int</code> ) of map coordinate XY                                                             |
| worldToMapValidated(wx, wy) | Get the mx ( <code>int</code> ), my ( <code>int</code> ) of map coordinate XY using wx ( <code>float</code> ) [m], wy ( <code>float</code> ) [m] of world coordinate XY. If wx wy coordinates are invalid, (None,None) is returned. |
| getIndex(mx, my)            | Get the index ( <code>int</code> ) of the cell using mx ( <code>int</code> ), my ( <code>int</code> ) of map coordinate XY                                                                                                          |

## Footprint Collision Checker API

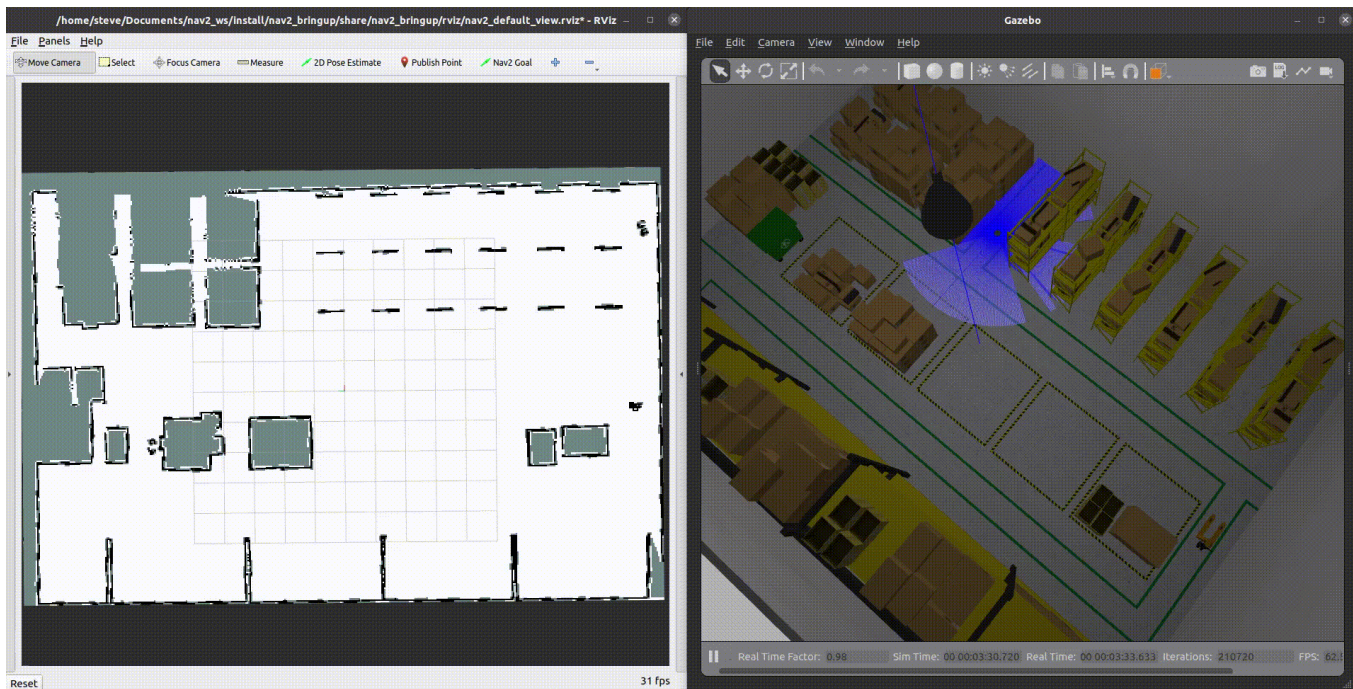
This is a Python3 API for a Footprint Collision Checker. It provides the needed methods to manipulate the coordinates and calculate the cost of a Footprint in a given map.

| Footprint Collision Checker Method      | Description                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| footprintCost(footprint)                | Checks the footprint ( <code>Polygon</code> ) for collision at its implicit provided coordinate pose.                                                                                                                                                                                                                        |
| lineCost(x0, x1, y0, y1, step_size=0.5) | Iterate over all the points along a line and check for collision. The line is defined by x0, y0, x1, y1, step_size ( <code>int</code> ) or ( <code>float</code> ).                                                                                                                                                           |
| worldToMapValidated(wx, wy)             | Get the mx ( <code>int</code> ), my ( <code>int</code> ) of map coordinate XY using wx ( <code>float</code> ) [m], wy ( <code>float</code> ) [m] of world coordinate XY. If wx wy coordinates are invalid, (None,None) is returned. Returns None if costmap is not defined yet through ( <code>setCostmap(costmap)</code> ). |

| Footprint Collision Checker Method                       | Description                                                                                                                               |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pointCost(x, y)</code>                             | Get the cost of a point in the costmap using map coordinates XY. ( <code>int</code> )                                                     |
| <code>setCostmap(costmap)</code>                         | Specify which costmap to use with the footprint collision checker. ( <code>PyCostmap2D</code> )                                           |
| <code>footprintCostAtPose(x, y, theta, footprint)</code> | Get the cost of a footprint at a specific Pose in map coordinates. x, y, theta ( <code>float</code> ) footprint ( <code>Polygon</code> ). |

## Examples and Demos

All of these can be found in the [package](#).



The `nav2_simple_commander` has a few examples to highlight the API functions available to you as a user:

- `example_nav_to_pose.py` - Demonstrates the navigate to pose capabilities of the navigator, as well as a number of auxiliary methods.
- `example_nav_through_poses.py` - Demonstrates the navigate through poses capabilities of the navigator, as well as a number of auxiliary methods.
- `example_waypoint_follower.py` - Demonstrates the waypoint following capabilities of the navigator, as well as a number of auxiliary methods.
- `example_follow_path.py` - Demonstrates the path following capabilities of the navigator, as well as a number of auxiliary methods like path smoothing.
- `example_assisted_teleop.py` - Demonstrates the assisted teleop capabilities of the navigator.
- `example_route.py` - Demonstrates the route server capabilities of the navigator.

The `nav2_simple_commander` has a few demonstrations to highlight a couple of simple autonomy applications you can build using the API:

- `demo_security.py` - A simple security robot application, showing how to have a robot follow a security route using Navigate Through Poses to do a patrol route, indefinitely.
- `demo_picking.py` - A simple item picking application, showing how to have a robot drive to a specific shelf in a warehouse to either pick an item or have a person place an item into a basket and deliver it to a destination for shipping using Navigate To Pose.
- `demo_inspection.py` - A simple shelf inspection application, showing how to use the Waypoint Follower and task executors to take pictures, RFID scans, etc of shelves to analyze the current shelf statuses and locate items in the warehouse.